

ANÁLISE COMPARATIVA DE ESTRATÉGIAS DE INSERÇÃO DE DADOS EM BANCOS RELACIONAIS UTILIZANDO PYTHON

Victor Augusto Soares de Oliveira

Universidade de Taubaté • Análise e Desenvolvimento de Sistemas • 2026
Orientador: Prof. Dr. Luis Fernando de Almeida

Migração de dados em sistemas de informação

A troca de sistemas exige preservar dados e continuidade da operação

- Ocorre em substituição ou modernização de sistemas.
- Dados do sistema legado precisam ser transferidos para o novo ambiente.
- A migração deve preservar integridade, consistência e disponibilidade.
- Em sistemas corporativos, o volume de dados pode tornar a carga um fator crítico.

Por que a migração exige planejamento

A execução da carga também impacta o sucesso da implantação

- Sistemas legados acumulam dados operacionais importantes.
- A carga precisa ocorrer dentro de uma janela aceitável.
- Falhas podem gerar inconsistência, retrabalho ou indisponibilidade.
- A estratégia de inserção influencia diretamente o tempo total.

Motivação:

A motivação surgiu de uma experiência profissional: a forma de inserir os dados pode impactar diretamente o tempo final da operação.

Problema investigado

O tempo de carga também pode comprometer a implantação

- Diferentes formas de inserir dados podem gerar tempos muito diferentes.
- Inserções linha a linha, lotes e cargas nativas têm custos distintos.
- A pesquisa compara essas estratégias em PostgreSQL e MariaDB.



Pergunta prática:

Qual estratégia insere 1 milhão de registros com menor tempo?

Objetivo e recorte

Comparar estratégias, não “provar” que um banco é melhor em qualquer cenário

1.000.000

registros sintéticos em CSV

2

SGBDR: PostgreSQL e MariaDB

4

estratégias de inserção

2

estruturas: com e sem *UNIQUE*

Recorte:

- ambiente local e controlado
- dados sintéticos
- tabela simplificada
- ausência de concorrência
- versões e configurações específicas

Desenho experimental

A comparação foi feita com as mesmas condições para cada banco



Cada cenário:

- 6 execuções sequenciais
- 1ª execução descartada como *warmup*
- 5 execuções válidas para média

Colunas da tabela

- | | |
|--------------|------------|
| • nome | • cidade |
| • e-mail | • estado |
| • CPF | • endereço |
| • data nasc. | • salário |

Estratégias avaliadas

O principal contraste está entre comandos repetidos, lotes e carga nativa

Individual transação única

1 INSERT por linha
commit só no final

Individual commit por registro

1 INSERT por linha
1 *commit* a cada linha

Inserção em lote

registros agrupados
em blocos

Carga em massa

COPY no PostgreSQL
LOAD DATA no
MariaDB

PostgreSQL: o custo do commit por registro dominou o tempo

COPY foi a melhor estratégia no PostgreSQL

2.143 s

*commit por registro, sem
UNIQUE*

181 s

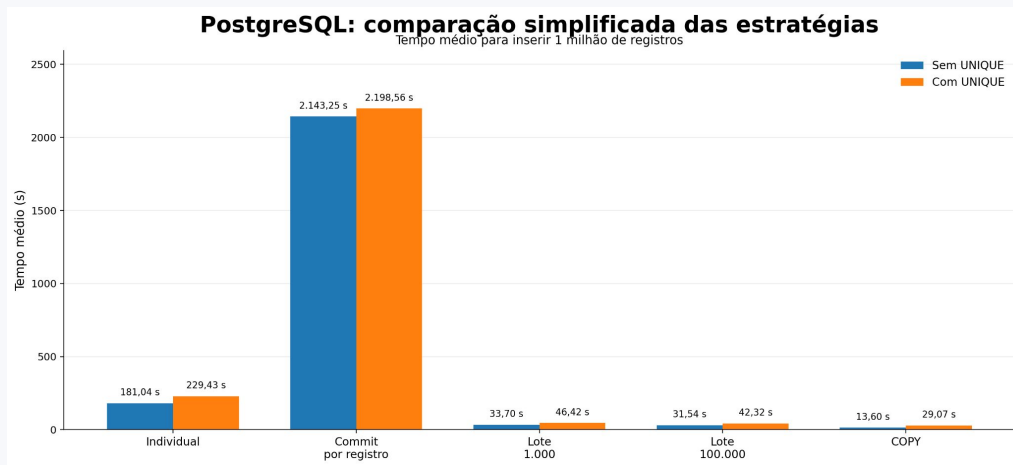
*individual, transação
única*

31 s

*lote 100.000, sem
UNIQUE*

13,6 s

COPY, sem UNIQUE



- *UNIQUE* aumentou o tempo das estratégias.
- Tamanhos de lote ficaram próximos entre si.
- COPY apresentou o menor tempo médio no PostgreSQL.

MariaDB: mesmo padrão, com menor tempo nas cargas rápidas

LOAD DATA foi a melhor estratégia no MariaDB

2.174 s

commit por registro, sem
UNIQUE

244 s

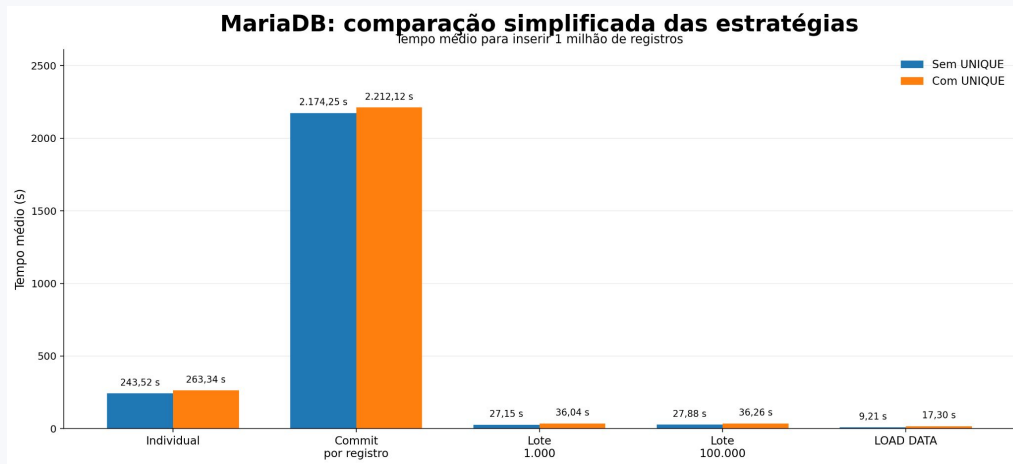
individual, transação
única

27 s

lote 1.000, sem *UNIQUE*

9,21 s

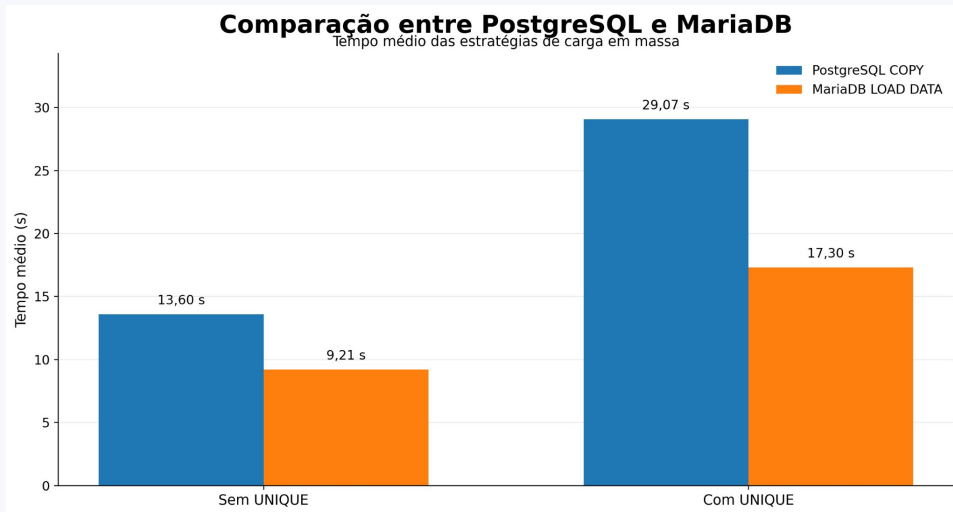
LOAD DATA, sem
UNIQUE



- *Commit* por registro também ficou acima de 2.100 s.
- Lotes ficaram próximos: cerca de 27 s sem *UNIQUE*.
- *LOAD DATA* chegou ao menor tempo geral do experimento.

Comparação entre PostgreSQL e MariaDB

O banco influenciou, mas a estratégia influenciou mais



MariaDB foi mais rápido neste ambiente, mas a conclusão principal não é “MariaDB sempre vence”.

A escolha da estratégia reduziu expressivamente o tempo nos dois bancos.

O que os resultados significam na prática

A diferença vem de transação, comunicação e manutenção de índices



Commit por registro

repete custo transacional
1 milhão de vezes



INSERT individual

reduz *commits*, mas
mantém um comando por
linha



Lote

reduz interações com o
banco e confirma em
grupo



Carga nativa

utiliza mecanismo nativo
otimizado para alto
volume



Ideia central:

Reduzir *commits* e interações
cliente-servidor reduz o custo
total da carga.

Conclusões principais

O que os resultados indicam sobre as estratégias de inserção

- *Commit* por registro foi a pior estratégia nos dois SGBDR.
 - Inserções em lote reduziram expressivamente o tempo de execução.
 - COPY e LOAD DATA foram as estratégias mais eficientes.
 - *UNIQUE* aumentou o custo de escrita por exigir manutenção de índices.
 - MariaDB foi mais rápido neste ambiente, mas a estratégia teve maior impacto que o banco.
-

Recomendação técnica:

Em cargas de grande volume, *commits* por registro devem ser evitados.

Obrigado!

Victor Augusto Soares de Oliveira

Universidade de Taubaté • Análise e Desenvolvimento de Sistemas • 2026
Orientador: Prof. Dr. Luis Fernando de Almeida